IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 20 August 2024, accepted 2 September 2024, date of publication 12 September 2024, date of current version 24 September 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3459656

RESEARCH ARTICLE

On the Effectiveness of Feature Selection Techniques in the Context of ML-Based Regression Test Prioritization

MD ASIF KHAN[1], (Member, IEEE), AKRAMUL AZIM[1], (Senior Member, IEEE), RAMIRO LISCANO[1], (Senior Member, IEEE), KEVIN SMITH[2], YEE-KANG CHANG[3], GKERTA SEFERI[2], AND QASIM TAUSEEF[2]

[1]Department of Electrical, Computer and Software Engineering, Ontario Tech University, Oshawa, ON L1G 0C5, Canada
[2]IBM United Kingdom Ltd., Portsmouth, PO6 3AU Hampshire, U.K.
[3]IBM Canada Ltd., Markham, ON L3R 9Z7, Canada

Corresponding author: Md Asif Khan (mdasif.khan@ontariotechu.net)

This work was supported in part by IBM Center for Advanced Studies (CAS) Grant, and in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) Grant.

ABSTRACT Regression testing is essential for maintaining software functionality in continuous integration (CI) systems, but it can become increasingly costly as software complexity grows. Machine learning-based Regression Test Prioritization (RTP) techniques have been developed to prioritize test cases based on their likelihood of failure, aiming to detect failures early and optimize resource use. However, the features used in the current state-of-the-art for training machine learning (ML) models often vary widely across different datasets, highlighting the need for further research to identify effective feature sets for RTP. Furthermore, the feature selection techniques are frequently biased toward specific features based on the dataset. Hence, we explored an ensemble technique to utilize three ML-based feature selection techniques in this study to identify and refine key features that enhance test case prioritization. These techniques were applied across four tree-based ML models using data from 15 large-scale open-source software projects. Our analysis identified the most compelling features for predicting failures and assessed their impact on RTP. The results showed that using a refined subset of features could achieve similar or up to a 10% increase in RTP performance, using only one-third of the original feature set. We also empirically evaluated the cost considerations when choosing the three methods and reported the ML models' performance with the refined feature sets. This underscores the potential of integrating advanced feature selection methods into RTP processes.

INDEX TERMS Continuous integration, feature selection, machine learning, test case prioritization.

I. INTRODUCTION

Continuous Integration (CI) has become a cornerstone for rapid and reliable code integration in software development. By automating the code integration and testing process, CI systems enable development teams to detect and correct failures early, accelerating the delivery of high-quality software. Regression Test Prioritization (RTP) is a technique that aims to optimize the order in which test cases are executed, to detect and resolve software failures quickly, which is essential for a well-functioning CI system. Hence, the significance of RTP cannot be exaggerated in an era where software development is increasingly characterized by complexity, rapid iteration, and the need for continuous feedback. The effective prioritization of test cases ensures that critical issues are promptly addressed, minimizing the cost and impact of defects during the software development life-cycle.

Machine Learning (ML) has recently witnessed significant advancements, exhibiting the potential to revolutionize various aspects of software engineering. In terms of enhancing the efficacy of software testing, it has demonstrated

The associate editor coordinating the review of this manuscript and approving it for publication was Sotirios Goudos.

promise in the context of RTP and has produced substantial results. The groundwork for using ML in RTP was laid by Elbaum et al. [1] by demonstrating its potential to reduce testing time and improve defect detection rates. ML has also shown adaptability to project-specific characteristics, resulting in more effective test case execution orders [2]. Furthermore, the superiority of ML algorithms in RTP compared to conventional methods has been emphasized in [3]. Empirical evidence of ML-based prioritization in web application testing has highlighted significant time savings and efficient defect detection [4]. Additionally, comprehensive overviews confirm that ML has become integral to contemporary RTP, revolutionizing software testing efficiency [5].

However, the performance and efficacy of ML-based RTP models are highly dependent on the selection and combination of input features, that are fundamental to ML models. Despite the growing interest in ML-driven RTP [6], more research is required to identify the specific features that enhance RTP performance. Additionally, potential sources for acquiring these features and the role of feature engineering in improving the RTP need to be explored further.

In a recent comprehensive study conducted by Yaraghi et al. [7], the most effective ML-based RTP techniques, particularly pairwise ranking with Random Forest (RF) [8], demonstrated 90% or higher effectiveness in only 5 out of the 25 studied projects. We argue that utilizing a single ML model, namely RF, for feature selection is the primary factor contributing to this limitation. Specifically, the authors identify 15 frequently used features out of 150 and categorize into three distinct feature groups. These features include test case size, development history, failure history, and age.

Yaraghi et al. [7] have utilized the RankLib tool [9] to calculate feature usage frequencies and assess the influence of specific features within their designated feature groupings. A higher frequency of utilization indicates that the feature tends to yield more significant knowledge gains throughout training. Nevertheless, our experimental findings revealed a bias in the frequencies of feature utilization in Random Forest (RF), specifically favoring Test Case Execution characteristics. This bias is distinct from that of previous ML models based on trees. Therefore, depending exclusively on the RF for RTP, the focus may be biased towards specific groups of features. This discovery implies investigate alternative tree-based machine learning models to achieve a more equitable and precise assessment of test case characteristics. To tackle this issue, we have employed four separate machine learning models, namely, RF, Decision Tree (DT), Gradient Boosting (GBoost), and eXtreme Gradient Boosting (XGBoost)-in addition to three strategies for determining the relevance of features to answer to following questions.

- **What are the top features identified through three different techniques?**
  The results indicate that the test verdict (Pass or Fail) of the last build and the failure rate over the last ten builds are the most significant features identified by the three feature selection techniques.
- **How do ML-based techniques trained on the top feature sets perform across subjects compared to models trained with all features in RTP?**
  ML models trained on the top ten features using Lasso-based feature selection techniques outperform those trained with all features in terms of RTP performance.
- **What are the prediction capabilities of ML-based feature selection techniques?**
  The impurity-based feature selection technique yields the highest values in terms of failure detection capability.
- **How do ML models trained with top features and full feature sets perform compared to heuristic models?**
  ML models outperform simple failure rate-based models 80% of the time, demonstrating improved RTP.

Our study provides several crucial insights into the application of ML-based feature selection techniques in RTP:

- Propose an ensemble technique to utilize three ML-based feature selection techniques to construct a minimal subset of features without compromising performance and minimizing the bias of individual methods.
- Our research Recommends a set of standard features readily obtainable from any CI dataset. This practical approach enhances the training of ML models, empowering practitioners in the field of RTP.
- Conduct extensive empirical analysis using 15 large-scale datasets to address four critical real-life questions.

The remainder of the paper is organized as follows: Section II discusses related work conducted in terms of features to improve RTP. Section III illustrates the architecture and approach of our case study and highlights the significance of applying different features to various classification techniques for failure detection and prioritization of test cases. This section also presents the research questions explored. Section IV presents our findings for the research questions and discusses the implications of our results in the context of improved RTP, followed by the disclosure of the validity of our research in Section V. Section VI concludes the paper.

II. RELATED WORK

The field of RTP has witnessed substantial research efforts, particularly in integrating ML techniques to enhance testing processes [10], [11], [12], [13]. In this section, we discuss prior research that leveraged features for RTP enhancement and highlight the gap that our study aims to fill, along with the limitations of existing work.

Lima et al. [14] and Khatibsyarbini et al. [15] showed that most studies in RTP utilize three primary sources of information: History-based information, Coverage-based information, and Others. History-based information includes failure history, which prioritizes tests likely to uncover faults based on past failures, and execution history, which focuses on the critical and frequently executed tests. Test age, another

history-based metric, refers to the age of test cases or the number of builds since their introduction, with older test cases often deprioritized in favor of newer ones that cover recent changes. Coverage-based information involves test coverage, indicating which parts of the code are covered by tests and statements, which counts the number of statements in the code covered by tests. Other significant sources include code complexity, which uses metrics to measure the complexity of the code being tested, and code metrics, such as cyclomatic complexity and code churn, to inform the prioritization process. We have summarized Table 1 to highlight the top sources for feature collection, as shown below.

TABLE 1. Primary information sources for feature collection used in RTP enhancement.

Category	Primary Studies
History-based	Ekinci et al. [16], Shakya and Briand [17], Khan et al. [18], Yoo et al. [19], Afzal and Torkar [20], Wang et al. [21], Bertolino et al. [22], Bagherzadeh et al. [23], Elsner et al. [24], Shakya and Briand [17], Spieker et al. [25], Marijan et al. [26], Afzal and Torkar [20], Bertolino et al. [22], Bagherzadeh et al. [23], Elsner et al. [24], Elsner et al. [24], Busjaeger and Xie [27]
Coverage-based	Yoo et al. [28], Elsner et al. [24], Jiang et al. [29]–[31], Elsner et al. [24]
Others	Ekinci et al. [16], Wang et al. [21]

Empirical studies, such as those by Shakya and Briand [17] and Khan et al. [18], have evaluated RTP techniques without primarily focusing on feature combinations. These studies, categorized under **History-based** for their use of failure and execution history, provided practical insights into the viability of prioritization techniques but did not extensively explore the potential enhancements achievable through feature selection. On the other hand, Yoo et al. [19] conducted a comprehensive survey of ML techniques for RTP enhancement, emphasizing data-driven techniques. This survey highlights the need to investigate how specific features, mainly from historical and coverage data, can be optimally combined to achieve superior results. Afzal and Torkar [20] made significant strides in applying ML to RTP by predicting the likelihood of failure based on historical data, fitting within the **History-based** category. However, their study predominantly focused on prediction and did not extensively explore the impact of feature combinations on prioritization effectiveness.

Wang et al. [21] combined historical test execution data with code metrics to predict likely failures. This approach, blending **History-based** and **Others** categories, shows the potential of combining diverse information sources. However, it is necessary to comprehensively investigate the influence of various feature combinations on RTP performance. Bertolino et al. [22] examined the performance of ten ML algorithms, including three reinforcement learning (RL) algorithms, for test prioritization in CI. They discovered that RL-based methods could better adapt to new changes with less sensitivity, with the Multiple Additive Regression Tree (MART) achieving the highest effectiveness. However, their study was limited by using a limited set of features based on the test cases' execution history, placing it within the **History-based** category. Bagherzadeh et al. [23] conducted an extensive analysis of ten advanced deep reinforcement learning techniques in the context of RTP, demonstrating that the best RL techniques could match the effectiveness of MART. However, this study also faced limitations similar to those of Bertolino et al., including a limited number of features and subjects.

Elsner et al. [24] explored a more comprehensive set of features, including **History-based** and **Coverage-based** for training ML models in RTP and found that features derived from the test execution history were more effective than others, confirming the importance of **History-based** features. They noted that simple heuristics based on the failure rate of test cases often outperformed complex ML models. However, their study was restricted to a subset of features, excluding critical features such as static coverage analysis and code complexity. Jiang et al. [30], [31] extensively utilized **Coverage-based** features such as functions, statements, and branches covered or functions, statements, and branches not covered yet. Still, they excluded history-based categories in their work. Additionally, Zhai et al. [29] prioritize test cases for regression testing of location-based services, considering **Coverage-based** features. Ekinci et al. [16] implemented search-based techniques, often employing heuristic functions based on code complexity and coverage, which provides a valuable foundation for our research. This study falls into the **Others** category as it utilizes code complexity metrics to guide the prioritization process.

Our study aims to address these limitations by conducting empirical research that explicitly examines the impact of diverse feature combinations on RTP performance. By exploring various ML-based feature selection techniques across **History-based**, **Coverage-based**, and **Others** categories, we seek to provide actionable insights into optimal feature combinations to enhance RTP effectiveness. Our study contributes to advancing RTP strategies and refining software testing practices through data-driven feature selection.

III. STUDY DESIGN

We outline the details of this study's design in this section.

A. FEATURE SELECTION TECHNIQUES

Selecting relevant features is crucial for training ML models in various applications, including failure prediction and RTP. Relevant features capture the essential characteristics of the software under examination, providing valuable insights into its behavior and stability. Features such as execution history, failure rates, and code change metrics enable ML models to learn patterns and correlations related to failure occurrences [32].

RTP aims to optimize the order in which test cases are executed to maximize the likelihood of detecting faults early in the testing process [33]. Relevant features, including test execution history and code change metrics, offer insights into software dynamics and guide the prioritization of test cases. However, including irrelevant features can lead to sub-optimal RTP performance, introducing noise into the model and making it challenging to prioritize the test cases effectively. These include reduced predictive accuracy, increased model complexity, and high resource consumption [34], [35]. A well-curated collection of relevant features streamlines the model training process, enabling efficient model training to facilitate faster iterations and experimentation, accelerating innovation and better prediction capability.

Hence, Feature selection plays a crucial role in improving the efficiency and effectiveness of ML models by identifying the most informative features for a given task. These techniques can be broadly categorized into two major groups: statistical analysis-based and machine learning-based.

1) STATISTICAL ANALYSIS-BASED METHODS

Statistical analysis-based methods evaluate the relationships between individual features and the target variables. These methods include univariate feature selection, which involves techniques such as the Chi-square test, ANOVA F-test, and Mutual Information. The Chi-square test is applicable for categorical target variables by measuring the dependence between categorical features and the target. The ANOVA F-test, on the other hand, is suitable for numerical target variables to assess the relationship between numerical features and the target. Mutual Information measures the dependency between variables and provides insights into the relevance of each feature. Additionally, correlation analysis is employed to identify and remove features that are highly correlated with each other to mitigate redundancy and multicollinearity issues. Domain knowledge is also leveraged to manually select or engineer features that are deemed relevant to a task.

2) MACHINE LEARNING-BASED METHODS

Machine learning-based methods incorporate the predictive power of ML models to evaluate and select features. These methods include impurity and gain-based techniques from tree based ML models, recursive feature elimination (RFE), principal component analysis (PCA), L1 (Lasso) regularization, embedded methods, feature selection with cross-validation, and forward/backward feature selection. impurity and gain values from trees quantifies the importance of features based on their contribution to reducing impurities in ensemble tree-based methods such as Random Forest and Gradient Boosting. RFE iteratively removes the least important features based on the model coefficients or feature importance until the desired number of features is reached. PCA is a dimensionality reduction technique that transforms features into orthogonal components ordered by the amount of variance explained, allowing for the selection of the top components that retain the most variance in the data. L1 (Lasso) regularization introduces sparsity by penalizing the absolute size of coefficients and effectively performing feature selection by setting some coefficients to zero. Embedded methods, such as Lasso Regression and tree-based models, inherently perform feature selection during training by favoring the most informative features. Feature selection with cross-validation conducts feature selection within each fold of cross-validation to ensure that the selected features are robust and generalizable. Forward/backward feature selection iteratively adds or removes features based on the model performance until a stopping criterion is met, starting with an empty set of features.

These feature selection techniques provide a systematic approach for identifying and retaining the most relevant features, thereby improving the model performance and interpretability in various machine learning tasks.

TABLE 2. Overview of feature selection techniques for RTP enhancement.

Category	Techniques
Statistical Analysis-Based	Chi-square test
	ANOVA F-test
	Mutual Information
	Correlation Analysis
	Domain Knowledge
Machine Learning-Based	Impurity and Gain from Trees
	Recursive Feature Elimination (RFE)
	Principal Component Analysis (PCA)
	L1 (Lasso) Regularization
	Embedded Methods
	Feature Selection with Cross-Validation
	Forward/Backward Feature Selection

These feature selection techniques provide a systematic approach for identifying and retaining the most relevant features, thereby improving the model performance and interpretability in various machine learning tasks.

In our study we are considering the following three ML-based feature selection techniques.

B. IMPURITY-BASED FEATURE IMPORTANCE (RANDOM FOREST, DECISION TREE AND GRADIENT BOOSTING)

Impurity-based feature importance is commonly used in tree-based ensemble methods such as Random Forest, Decision Tree and Gradient Boosting. We used it to measure the contribution of each feature to the reduction in impurity [36] or criterion (usually Gini [37] impurity or entropy) in the decision trees.

Mathematically, for a given feature X_i in a decision tree or ensemble of trees,

$$\text{Gini Importance}(X_i) = \sum_{\text{nodes}} \text{Gini Impurity Before Split} - \text{Gini Impurity After Split}$$

$$\text{Impurity Reduction}(X_i) = \sum_{\text{trees}} \frac{N_i}{N} \cdot \text{Impurity Before Split} - \text{Impurity After Split}$$

where:

N_i : Number of samples at the node where the feature is used.

N : Total number of samples in the dataset.

C. PERMUTATION-BASED FEATURE IMPORTANCE (XGBOOST WITH TYPE GAIN)

The Permutation-based feature importance in XGBoost evaluates the importance of a feature by permuting its values and measuring the drop in model performance (e.g., decrease in accuracy or increase in loss). A larger drop in the performance indicates a more important feature.

The metric "Type Gain" refers to the use of gain (as explained above) as the performance metric when assessing the feature importance in XGBoost.

Mathematically, the permutation-based importance score for a feature X_i in XGBoost with Type Gain is calculated by comparing the model's performance metric (e.g., gain) before and after permuting the values of X_i: Permutation-based Importance(X_i):

$$= \text{Performance Metric Before Permutation} - \text{Performance Metric After Permutation}$$

D. PRINCIPAL COMPONENT ANALYSIS (PCA) BASED FEATURE IMPORTANCE

Principal Component Analysis (PCA) is a dimensionality reduction technique that is widely used for feature selection in machine learning and data analysis. By transforming high-dimensional data into a lower-dimensional space, PCA retains most of the essential information while discarding redundant and noisy features. The key idea behind PCA is to identify the directions (principal components) in which the data vary the most and project the original data onto these components. This allows for a more compact representation of the data, making it easier to visualize and analyze. PCA is particularly useful when dealing with datasets with a large number of features because it helps mitigate the curse of dimensionality by focusing on the most informative dimensions.

Mathematically, the equation for PCA can be summarized as:

$$X' = X \cdot \mathbf{V}$$

where: - X' is the transformed dataset. - X is the standardized original dataset. - $\mathbf{V}$ is a matrix containing the selected eigenvectors as its columns.

One of the main advantages of PCA is its ability to reduce the data complexity, while preserving important patterns and structures. PCA can effectively reduce the dimensionality of the dataset by selecting a subset of the principal components that capture most of the variance in the data. This not only speeds up computation but also improves model performance by reducing overfitting and enhancing generalization. However, it's important to note that PCA has some limitations. For example, PCA assumes linear relationships between variables and may not perform well on nonlinear data. Additionally, interpreting the resulting principal components in terms of the original features can be challenging, particualarly when dealing with high-dimensional datasets. Despite these limitations, PCA remains a powerful tool for feature selection and dimensionality reduction for a wide range of applications.

E. L1 REGULARIZATION (LASSO)

L1 regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator), is a technique used in machine learning and statistics for feature selection and regularization. It adds a penalty term to the cost function, which penalizes the absolute magnitude of the feature coefficients.

In the context of linear regression, the L1 regularization term is added to the ordinary least squares (OLS) cost function. The modified cost function with L1 regularization penalty is given by:

$$J(\mathbf{w}) = \text{MSE}(\mathbf{w}) + \lambda \sum_{i=1}^{n} |w_i|$$

where:

- $J(\mathbf{w})$ is the regularized cost function.
- MSE($\mathbf{w}$) is the mean squared error (MSE) of the model.
- $\mathbf{w}$ is the vector of model coefficients.
- λ is the regularization parameter, which controls the strength of the regularization.
- n is the number of features.

One of the key advantages of L1 regularization is its ability to perform automatic feature selection by forcing some coefficients to become zero. This sparsity-inducing property of L1 regularization makes it particularly useful for high-dimensional datasets with many irrelevant or redundant features. Features with zero coefficients are effectively removed from the model, thereby simplifying it and improving its interpretability.

L1 regularization, or Lasso, is widely used in various ML algorithms, including linear regression, logistic regression, and support vector machines (SVM). It is particularly popular in situations where feature selection and model interpretability are important considerations.

F. ML MODELS

To train our models, we used all available features with four different ML models:

1) Decision Trees (DT),
2) Random Forest (RF),
3) Extreme gradient boosting (XGBoost), and
4) Gradient Boosting (GBoost)

Our research prioritized tree-based ML models due to their proven high performance in RTP. This decision is supported by numerous studies, including Menzies et al. [38], which emphasize the robustness and capability of tree-based models

to capture intricate relationships within software testing data. Additionally, Bertolino et al. [22] demonstrated that Multiple Additive Regression Trees (MART) and gradient-boosted regression trees are among the most effective ML models for RTP ranking. Contrarily, Yaraghi et al. [7] found that the Random Forest (RF) model outperformed others, including MART, leading them to rely on RF for their extensive testing. Factors such as the interpretability of tree-based models, their ability to handle both categorical and numerical features, and their effectiveness in determining feature importance played crucial roles in our choice [39]. The primary aim of this study is to evaluate the impact of various feature selection techniques using these models to enhance RTP.

G. PERFORMANCE METRICS

In the performance metrics subsection, we focus on evaluating RTP models using the following metric:

- Precision, Recall, F1-Score, and AUC: These metrics [12], [40] provide a comprehensive assessment of model performance, including precision (positive predictive value), recall (true positive rate), F1-Score (harmonic mean of precision and recall), and AUC (area under the receiver operating characteristic curve) [14], [41]. These metrics are crucial for understanding the model's effectiveness in test case prioritization and provide a well-rounded view of the RTP performance.
- Average Percentage of Faults Detected (APFD): This metric [1], [42], [43] quantifies the effectiveness of different feature combinations for improving RTP. The APFD assesses how well a given test case prioritization strategy detects faults by considering both the order and actual faults detected. A higher APFD score indicates a more effective RTP strategy.
- Average Failure Position (AFP): This metric assesses the effectiveness of a prioritization technique in reordering test cases based on their likelihood of failure. The AFP metric is calculated by determining the index positions of the failed test cases within a prioritized test suite and then computing the mean value of these positions.
  A lower AFP value indicates that the prioritization method is successfully identifyies and places test cases that are more likely to fail at the beginning of the test execution queue. This improvement in test case prioritization can lead to faster fault detection and potentially more efficient testing procedures.

By incorporating these performance metrics, we aim to provide a thorough evaluation of the RTP models, enabling us to assess the impact of different feature combinations on their effectiveness in detecting faults and optimizing test case prioritization strategies across various software projects.

H. RESEARCH QUESTIONS

This study investigates ML-based feature selection techniques in the context of CI from three primary perspectives: effectiveness, efficiency, and applicability. Specifically, we address several research questions to understand the performance and utility of these techniques comprehensively. First, we aim to identify the optimal features extracted using diverse ML-based feature selection methods (RQ1). Subsequently, we evaluate the performance of these techniques across different subjects and CI cycles, seeking insights into their efficacy and consistency (RQ2 and RQ3 respectively).

- **RQ1. What are the top features obtained through three different feature selection techniques, and what are the common features when combined?**
  This research question examines the best features obtained using different ML-based feature selection techniques. To answer this question, we collected the top ten features for each feature selection technique and combine them to generate a full list. Answering this question can help us choose a minimal number of features when training ML models for RTP.
- **RQ2. How do the ML-based techniques trained in the top feature sets perform across subjects compared to the models trained with all features?**
  This research question, with its potential benefits, seeks to determine if ML-based feature selection techniques show varying performance levels across different subjects and to understand the reasons behind any observed disparities. To address this, we calculate the APFD results for the feature selection techniques applied to each subject. The findings from this analysis can guide the selection of the most suitable feature selection technique for different subjects, thereby enhancing the efficiency and effectiveness of feature selection in machine learning.
- **RQ3. How do the prediction capability and time efficiency aspects of ML-based feature selection techniques compare in RTP, focusing on training time and state-of-the-art evaluation metric such as precision, recall, F1 score and AUC ?**
  This research question explores the efficiency of ML-based feature selection techniques in terms of time expenditure. By examining the time required for feature selection, model training, and prediction, we aim to determine which approach offers the most efficient utilization of time resources. Answering this question will provide valuable insights into selecting the most time-effective technique to expedite fault detection during RTP testing.
- **RQ4. How do ML models train with top features and full feature sets perform compared with heuristic models?** This research question compares the performance of machine learning models trained with the top features extracted through feature selection techniques and models trained with the full feature set against heuristic models. By evaluating metrics such as accuracy, precision, and recall, we aim to discern the efficacy of feature selection in enhancing fault detection during testing compared with traditional heuristic approaches.

I. SUBJECTS

Yaraghie et al. [7], Mendoza et al. [44], and Elsener et al. [24] conducted extensive examinations of ML-based techniques using 25 datasets. Each dataset was meticulously crafted with a comprehensive set of 150 features, making them some of the most thorough and publicly accessible datasets. These datasets were derived from analyzing over 20,000 open-source Java projects with five-star ratings. The inclusion criteria ensured that all subjects had a sufficient number of failed tests and a regression testing time of more than 5 minutes, both critical for applying and evaluating RTP techniques. The datasets also represent a wide range of source code sizes, from approximately 60k to 4.6M lines of code, with a median of roughly 230k. They cover the number of tests in each continuous integration (CI) build, ranging from approximately 30 to 4400, with a median of 117, and the number of failed builds, which range from 6 to 343, with a median of 83. Additionally, we created another large-scale CI dataset from IBM Open Liberty, a fast and composable Java-based application server used for developing applications and cloud-based services [18].

We further refined the dataset as smaller datasets may result in unstable results, while large-scale CI systems likely contain more failed tests than passed tests. Hence, the datasets were chosen with a running period of over 200 days to mitigate the poor training phase. Following these selection criteria, we intended to secure a collection of datasets, as illustrated in Table 3, which embodies diversity, public accessibility, and authenticity in representing real-world software-testing scenarios. We also added the Open Liberty dataset provided by the IBM.

By adhering to these selection criteria, we intended to ensure that the selected datasets listed in Table 3 were diverse, publicly accessible, and representative of real-world software testing circumstances. This methodology strengthens the validity of our findings and enables comparisons with other studies in the field.

J. FEATURE COLLECTION

A ML-based RTP model takes in feature vectors of test cases and ranks these instances accordingly. A feature must relate to any attribute of the test instances to be employed in training. This often requires aggregating and reformatting data collected at different levels of detail.

- **Age**: This feature represents the count of builds that have occurred since the initial execution of the test case. The age of test case t at build n is determined by subtracting the first build i from the current build n. This feature is based on prior research [27], which suggests that newer test cases are more likely to fail due to their interaction with recently added or modified code.
- **LastFailAge**: This feature quantifies the number of builds that have occurred since the most recent failure of the test case. The formula to get the *LastFailAge* for test case t at build n is $n-i-1$, where i is the build number on which the most recent failure of test case t occurred. The value of *LastFailAge* is set to -1 for test cases that have never failed, while a test case that failed in the previous build has a *LastFailAge* of 0.
- **LastTransitionAge**: This attribute quantifies the number of builds that have occurred since the most recent alteration in the test case's result, regardless of whether it was a pass or a fail. The calculation of this value is analogous to that of *LastFailAge*, except it emphasizes explicitly the most recent occurrence of transitioning from a pass to a fail verdict.
- **AvgExeTime**: This feature represents the mean duration of the test case's execution in prior iterations.
- **MaxExeTime**: This characteristic denotes the highest documented execution duration for the given test case.
- **FailRate**: The characteristic is represented by the ratio f/n, where f denotes the number of unsuccessful executions of the test case, and n represents the total number of executions. The raw failure count might be misleading as it depends on the frequency of test case execution. Therefore, this statistic is favored over the raw failure count.
- **AssertRate**: This feature measures the frequency at which the test case fails because of assertion failures. Assertion failures happen when the output of the system under test (SUT) does not match the intended conclusion.
- **ExcRate:** This feature quantifies the frequency of exception failures, which occur when exceptions are not adequately managed within the code of the test case. Exception failures frequently suggest problems inside the test case rather than the System Under Test (SUT), distinguishing them from assertion failures
- **TransitionRate**: This feature quantifies the rate at which the test case's verdict switches between pass and fail.
- **LastVerdict**: This functionality captures the result (either pass or fail) of the most recent execution of the test case.
- **LastExeTime**: This feature records the duration of the test case's most recent execution.
- **Line Inserted and deleted**: These features monitor the number of code lines added and removed in each commit.
- **Duration**: the time each test case takes to execute under each build.

CI systems execute regression test cases regularly, leading to a significant increase in execution history. Performance indicators like AvgExeTime, MaxExeTime, FailRate, AssertRate, ExcRate, and TransitionRate are calculated based on logs from the most recent n test case executions. Two values are computed for these six features: **Recent** and **Total**. Recent values are derived from the six most recent builds as suggested in [25], while Total values are computed from all available builds.

TABLE 3. Statistics of the 15 selected subjects.

Subject	SLOC	Java SLOC	# Commits	Time Period (months)	# Builds	# Failed Builds	Build Fail Rate (%)
apache/curator	84k	58k	3.1k	21	517	65	12
apache/rocketmq	135k	100k	2.0k	16	536	56	10
apache/shardingsphere	422k	165k	29.6k	7	1,049	123	11
apache/sling	695k	388k	47.4k	7	1,403	343	24
camunda/camunda-bpm-platform	653k	395k	20.6k	34	822	125	15
eclipse/steady	221k	98k	2.0k	13	675	51	7
EMResearch/EvoMaster	158k	25k	4.1k	7	583	109	18
facebook/buck	586k	384k	26.3k	10	846	130	15
Graylog2/graylog2-server	182k	85k	22.3k	53	3,668	124	3
IBM Open-Liberty	100k	25k	39k	39	6,124	17.5k	0.38
jcabi/jcabi-github	61k	32k	2.8k	29	788	6	< 0.01
JMRI/JMRI	4.56M	1.05M	69.3k	5	1,481	65	4
SonarSource/sonarqube	899k	398k	31.8k	18	4,286	230	5
yamcs/Yamcs	229k	123k	5.6k	24	504	61	12
zolyfarkas/spf4j	125k	79k	32.6k	37	587	180	30

Algorithm 1 Feature Collection & Selection Technique for RTP

Require: Continuous Integration dataset D_{CI}, Code change-based dataset D_{CC}
Ensure: Top Features F_{top} for RTP

```
D_CI ← Collect CI dataset
D_CC ← Collect code change-based dataset from GitHub
Commits ← Query GitHub for all commits under a certain Git head
for each CI job in D_CI do
    ParentID ← Retrieve parent job ID
    CI_info ← Merge CI information with GitHub information
    GitHead ← Retrieve the Git head commit
    Diff ← Get the diff of changes from GitHub
    ChangedFiles ← Retrieve information of changed files for all commits of parent jobs
    TestBuckets ← Map changed files with test buckets
end for
D'_CI ← Filter D_CI to refine dataset
D'_CC ← Filter D_CC to refine dataset
F ← Perform feature engineering to construct feature set from D'_CI and D'_CC
F_PI ← Apply Impurity or Gain-based feature selection to F
F_PCA ← Apply PCA-based feature selection to F
F_Lasso ← Apply Lasso-based feature selection to F
F_top ← F_PI ∪ F_PCA ∪ F_Lasso ▷ Combine feature subsets
return F_top ▷ Return the top features for RTP
```

K. IMPLEMENTATION

Algorithm 1 outlines a systematic approach for identifying critical features from software development data to enhance test case prioritization in Continuous Integration (CI) environments. The following is an explanation of each step:

1) **Data Collection:** The algorithm begins by gathering data from two primary sources: a Continuous Integration (CI) dataset (D_{CI}) and a code change-based dataset (D_{CC}) from GitHub. This includes details of the CI jobs and commits linked to specific code changes.
2) **Integration of Data:** For each CI job in D_{CI}, relevant information such as parent job IDs, Git head commits, diffs of changes, and lists of files affected by commits are retrieved and combined. This step maps these changes to specific test cases or buckets.
3) **Data Refinement:** Raw datasets D_{CI} and D_{CC} are filtered to refine the data, focusing on the most pertinent details that could influence test case prioritization.
4) **Feature Engineering:** A comprehensive feature set (F) is constructed using feature engineering techniques on refined datasets. This includes deriving statistical, structural, and historical features that may affect the CI processes.
5) **Feature Selection Techniques:** Three different feature selection techniques are applied to the engineered features:
   Impurity and Gain-based (IG) feature selection: After implementing this technique as discussed in Section III-B, we use the APFD (Average Percentage of Faults Detected) as the determinant. We iteratively add features based on their IG values and observe the APFD value. If the APFD value does not increase after adding a feature, that feature is dropped. We found that features with an IG value below 0.05 contribute minimally to APFD performance. Studies such as Prasetiyowati et al. [45] have also shown that a cut-off value of 0.05 effectively balances feature selection, retaining significant features while discarding less impactful ones.
   Principal Component Analysis (PCA) based feature selection: For the PCA-based feature selection, we observe the impact of each principal component

on the APFD value. Similar to the IG approach, we retain components that contribute to an increase in APFD. Typically, components explaining up to 95% of the variance are retained, as features contributing below this threshold generally add minimal value. This approach aligns with the findings of Zhao et al. [46], who demonstrated the effectiveness of using a 95% variance threshold to preserve essential data characteristics while reducing dimensionality.

Lasso-based (Lasso) feature selection: This technique inherently assigns a weight of 0 to the least important features, effectively excluding them from the model as discussed in Section III-E. This approach does not require a manually set threshold, as the regularization process naturally determines the significance of features based on their coefficients. We observed that the features discarded by this method often have weights of 0, ensuring that only the most relevant features are included.

6) **Combination of Selected Features:** The top features from each selection method are combined to form the final set of top features (F_{top}), ensuring that the features are robust and capture various data aspects that are critical for effective test prioritization.
7) **Output:** The algorithm outputs F_{top}, the set of top features determined to be the most effective for prioritizing test cases based on the observed historical and current data patterns in the CI and code change datasets. Additionally, we keep the record of the top features from individual techniques

This methodical approach as shown in Figure 1 is designed to ensure that prioritized test cases are those most likely to capture and address potential faults introduced by recent code changes, thereby improving the efficiency and effectiveness of the testing processes in CI environments.

L. WORKFLOW

We start by collecting feature sets from two sources: CI test execution results and the version control system (VCS). In the feature mapping step, we combine the CI test results with related commits to find changes in the VCS. We further filter and clean the data in the data processing step, which provides us with a usable feature set to train the ML models.

We use the scikit-learn built-in Python packages to implement all the ML models and ML-based feature selection techniques. For this study, we use the default hyperparameters for each model. Hyperparameter tuning is a separate task that could significantly impact the performance of the ML models. Still, it adds computational overhead, especially with an extensive feature set and various feature combinations. The main goal of this paper is to explore how different feature sets obtained through different machine-learning techniques affect RTP, so keeping the models in their default form was a design choice. However, we acknowledge that tuning hyperparameters can affect the performance of machine learning models.

We retrain the models for each new feature set to compare how the RTP value changes for different feature sets. There is no universal method to estimate the ratio for splitting a dataset into training and testing sets or determining the training data needed to create an effective model. One study [47] used one year of data to train the model and then assessed it using the latest two months of data. The efficacy of the prediction model diminishes over time, prompting regular updates. This paper describes a system that undergoes periodic offline training and utilizes all the accessible historical data for training purposes. Owing to the temporal dependence during the intervals between CI cycles, it is not feasible to perform k-fold cross-validation.

Consequently, it is more pragmatic to train models on historical data and experiment with more current data [47]. Hence, we perform an approximate 60/40 train-test split on 15 datasets by choosing a particular build date before which 60% of the build results exist. Then, we employ four machine learning models: DT, RF, XGBoost, and GBoost. All available features from the feature collection section are used. The models are trained on the training data, and their performance is evaluated on the testing data. Key evaluation metrics are recorded, including the APFD, Precision, Recall, F1-Score, and AUC.

In summary, the workflow pipeline begins with data collection, gathering historical test execution data, failure history, and code metrics. Next, we perform feature selection using various ML techniques. This process is thorough, ensuring that we identify the most relevant features for RTP. The selected features are then used to train the ML models. After training, the models are tested on the testing dataset, and their performance is evaluated using the mentioned metrics. The process ensures that the models are continually updated with the latest data to maintain effectiveness.

IV. RESULTS AND ANALYSIS

In this section we present our findings to answer the research question.

A. RQ1: IMPACTFUL FEATURES

1) IMPURITY AND GAIN-BASED FEATURE SELECTION USING TREE-BASED ML MODELS

To answer RQ1, we run the four different ML models for the fifteen datasets and record the features with the highest impurity and Gain (IG) value, as shown in Table 4. The numerical values in the parenthesis adjacent to each feature name indicate the impurity (DT, RF, GBosst) or Gain value (XGBoost only) calculated using the criteria described in Sections III-B and III-C. These values indicate which feature is most helpful in predicting failed test suites. We hypothesize that certain features are more valuable than others at prediction failed tests and will also appear frequently as top features in different datasets.

First, we analyze the top features within the four different ML models and notice subtle differences between the top

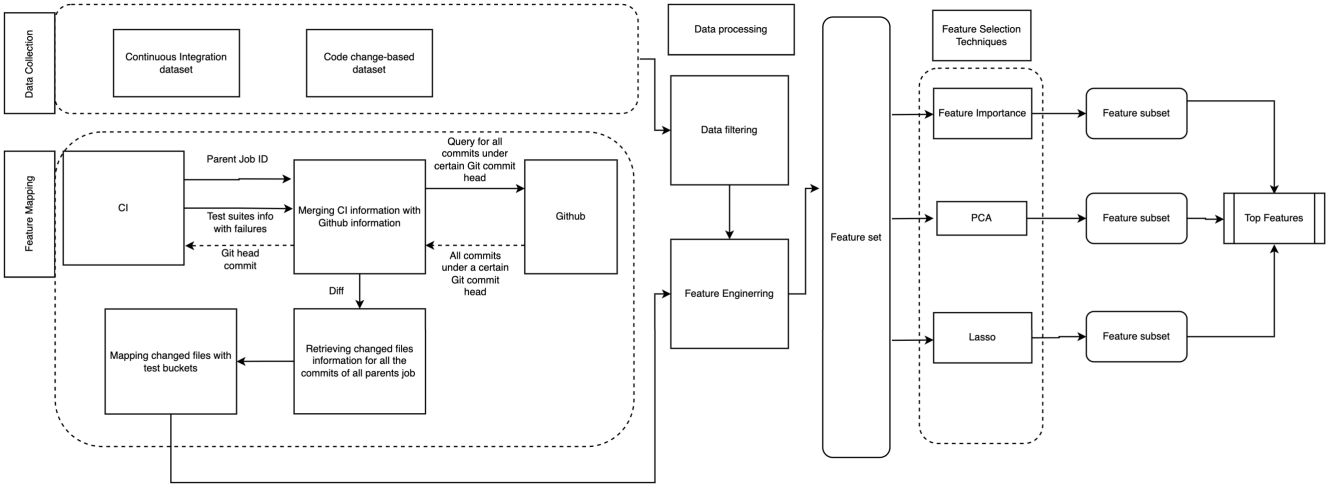

FIGURE 1. Workflow diagram (Left to right) of the proposed feature selection technique.

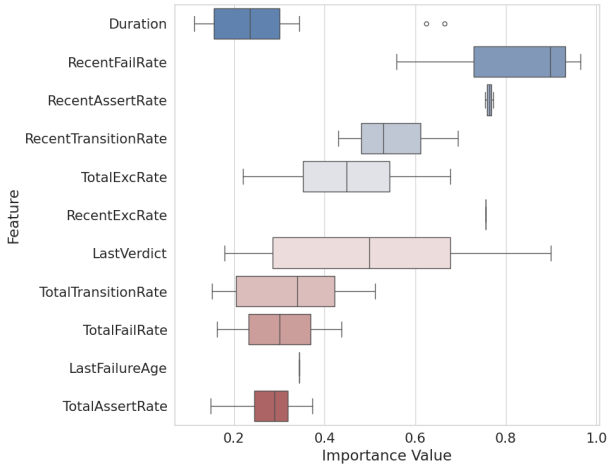

FIGURE 2. Impurity and Gain-based (IG) value of the top features.

features. For the DT model, *RecentFailRate* stands out with the highest IG value value of 0.964 for the dataset *Facebook/buck*. In contrast, the RF model assigns the highest IG value to *Duration*, with a significance value of 0.235, observed for datasets *camunda/camunda-bpm-platform* and *facebook/buck*. The XGBoost model attributes the most significant IG value to *LastVerdict*, showcasing a peak value of 0.899 for dataset *SonarSource/sonarqube*, and similarly, for dataset *Facebook/buck*, this feature also prevails with the highest IG value of 0.845 in GBoost for the *SonarSource/sonarqube* dataset. When we analyze the datasets that produce high IG value values for particular features, we realize that all the high values have occurred on the top 5 largest datasets of our subjects.

Finding 1: In the larger dataset (SLOC > 500K) tree-based IG techniques find *RecentFailRate* and *LastVerdict* features most useful to predict failed test cases.

Then, we investigate the IG value value across all the datasets in Figure 2 with the provided IG values. The features *RecentFailRate* and *LastVerdict* exhibit notable fluctuations in their IG value values across classifiers. *RecentFailRate*, in particular, displays substantial variation from its median value (> 0.8), suggesting a lack of consistency in its evaluated gain values. This variability underscores the feature's significance as being highly classifier-dependent. Conversely, *Duration* manifests relatively minor deviations from its median IG value value (> 0.3), underscoring a higher level of consensus among classifiers regarding its significance. The confined interquartile range and minimal outliers for *Duration* reflect a broad agreement on its relevance across the evaluated machine learning models. Additionally, the features *RecentAssertRate* and *TotalExcRate* show more consistent values, as evidenced by their narrower box plots and medians that are closely aligned with other classifiers. The consistent nature of these qualities indicates a widespread agreement on their ability to predict failed tests, establishing them as reliable features within the dataset.

Finding 2: *RecentFailRate* and *LastVerdict* features have the highest fluctuations while *RecentAssertRate* and *RecentExcRate* have the least fluctuation in their IG values across 15 datasets.

However, we understand that the occurrence of the features are not equally distributed among the datasets; for example *RecentAssertRate* occurs only 3 times but shows high IG value range. Hence, we conduct a more detailed examination of the IG value of these features by measuring how frequently they appear in the datasets. Our hypothesis suggests that features commonly considered most important have a more significant influence than other features. To verify this, Table 5 catalogs the frequency with which each feature attains the highest IG value under various ML models, arranging them in descending order for clarity. This organization ensures that the feature with the highest frequency positions itself at the summit.

Our findings reveal a notable disparity: *Duration* emerges as the preeminent feature, claiming the highest IG value a striking 21 times across all evaluated datasets. Meanwhile,

TABLE 4. Features with highest IG values across four classifiers in 15 datasets.

Subjects	DT	RF	XGBoost	GBoost
apache/curator	TotalAssertRate(0.277)	Duration(0.117)	TotalFailRate(0.437)	Duration(0.236)
apache/rocketmq	TotalTransitionRate(0.378)	TotalTransitionRate(0.172)	TotalFailRate(0.437)	Duration(0.223)
apache/shardingsphere	RecentExcRate(0.756)	TotalExcRate(0.220)	TotalFailRate(0.437)	TotalExcRate(0.677)
apache/sling	RecentTransitionRate(0.530)	TotalFailRate(0.163)	RecentFailRate(0.694)	RecentFailRate(0.430)
camunda/camunda-bpm-platform	Duration(0.665)	**Duration(0.235)**	TotalExcRate(0.499)	TotalExcRate(0.397)
eclipse/steady	LastVerdict(0.498)	LastVerdict(0.179)	LastVerdict(0.621)	LastVerdict(0.483)
EMResearch/EvoMaster	Duration(0.665)	Duration(0.184)	LastVerdict(0.225)	Duration(0.344)
facebook/buck	**RecentFailRate(0.964)**	**Duration(0.235)**	**LastVerdict(0.898)**	Duration(0.247)
Graylog2/graylog2-server	TotalTransitionRate(0.511)	TotalFailRate(0.151)	TotalFailRate(0.437)	Duration(0.300)
IBM Open-Liberty	LastFailureAge(0.344)	Duration(0.138)	LastVerdict(0.566)	RecentFailRate(0.301)
jcabi/jcabi-github	RecentAssertRate(0.754)	TotalAssertRate(0.149)	LastVerdict(0.734)	TotalAssertRate(0.373)
JMRI/JMRI	Duration(0.328)	Duration(0.112)	RecentFailRate(0.559)	Duration(0.248)
SonarSource/sonarqube	RecentAssertRate(0.772)	Duration(0.201)	**LastVerdict(0.899)**	**LastVerdict(0.845)**
yamcs/Yamcs	Duration(0.202)	Duration(0.121)	Duration(0.624)	Duration(0.202)
zolyfarkas/spf4j	LastVerdict(0.345)	Duration(0.156)	LastVerdict(0.197)	Duration(0.300)

TABLE 5. Frequency of features with highest IG for each ML model.

Feature	DT	RF	XGBoost	GBoost	Total Count
Duration	4	9	1	8	22
LastVerdict	2	1	7	2	12
RecentFailRate	1	0	5	3	9
TotalFailRate	0	3	3	2	8
RecentAssertRate	1	1	1	0	3
TotalAssertRate	0	1	1	1	3
RecentTransitionRate	1	0	1	1	3
TotalExcRate	0	1	1	1	3
RecentExcRate	1	0	0	1	2
TotalTransitionRate	1	1	0	0	2
LastFailureAge	1	0	0	1	2

LastVerdict secures its position 12 times. A particularly compelling observation is the distribution of IG value within individual models. For instance, the RF model awards *Duration* the pinnacle of IG value in 9 out of the 15 datasets, significantly contributing to its overall dominance. Conversely, XGBoost prefers *LastVerdict*, selecting it as the most critical feature in 7 out of 15 datasets.

The results of our study show a significant difference: the feature *Duration* stands out as the most important, being identified as the highest priority 21 times in all the datasets analyzed. Meanwhile, *LastVerdict* maintains its place 12 times. An intriguing observation regards the allocation of significance within individual models. For example, in 9 out of the 15 datasets, the RF model assigns the highest level of relevance to the variable called *Duration*, significantly influencing its overall dominance. In contrast, XGBoost prefers *LastVerdict*, identifying it as the most crucial feature in 7 out of 15 datasets.

Finding 3: In term of frequency, the *Duration* and *LastVerdict* features appears more than 50% times as the top features across 15 datasets.

2) PCA AND LASSO-BASED REGRESSION FEATURE SELECTION

In this section, we input all our features in PCA and Lasso analysis; the output of these techniques are a subset of input features. The features obtained through IG technique from Table 5 are compared using Lasso and PCA feature selection techniques in Table 6. An exhaustive analysis of the Lasso-based feature selection shows an apparent preference for prioritizing execution time and variables linked to age. The features *Age*, *RecentAvgExeTime*, *RecentMaxExeTime* along with the *TotalAvgExeTime*, *TotalMaxExeTime*, and *Duration* are chosen explicitly in the Lasso-based feature selection training data, indicating their significant impact on the model's predictive accuracy. Furthermore, the feature *LastFailureAge* is noteworthy because of its occurrence in the Lasso and PCA based feature selection training sets, indicating its well-recognized importance. Meanwhile, the features *CommitCount* and *LinesAdded*, together with *LinesDeleted*, are only found in the Lasso-based feature selection, suggesting that they may be helpful for training models in this particular situation.

The comparison of the selected features in Lasso and PCA training data demonstrates contrasting methods of feature selection, which represent the underlying methodologies and objectives of these two techniques. As discussed in Section III-D, PCA approach does not explicitly choose individual features but instead integrates them into primary components. Still, the fact that *LastVerdict* and *TotalFailRate* are in the PCA subset suggests that they depend on parts that capture a lot of variation, which may be linked to these features. Unlike Lasso-based feature selection, PCA can combine information from various features, particularly associated ones, into principal components. This is seen in

TABLE 6. Frequency of features for three feature selection techniques.

Feature	Feature Selection Techniques			Frequency
	Lasso	PCA	IG	
LastFailureAge	✓	✓	✓	3
LastVerdict	-	✓	✓	2
RecentAssertRate	-	✓	✓	2
RecentExcRate	-	✓	✓	2
RecentFailRate	-	✓	✓	2
RecentTransitionRate	-	✓	✓	2
TotalAssertRate	-	✓	✓	2
TotalExcRate	-	✓	✓	2
TotalFailRate	-	✓	✓	2
TotalTransitionRate	-	✓	✓	2
TotalAvgExeTime	✓	✓	-	2
TotalMaxExeTime	✓	✓	-	2
LinesAdded	✓	✓	-	2
CommitCount	✓	✓	-	2
Duration	✓	-	✓	2
Age	✓	-	-	1
LastExeTime	✓	-	-	1
LastTransitionAge	✓	-	-	1
RecentAvgExeTime	✓	-	-	1
RecentMaxExeTime	✓	-	-	1
LinesDeleted	✓	-	-	1

the vast array of features identified for PCA-based feature selection, encompassing failure rates, assertion rates, and execution characteristics. Moreover, this technique performs feature transformation instead of feature selection, which could account for the broader range of necessary features. One interesting observation is that the PCA approach emphasizes features that substantially impact the variance in the dataset, such as *RecentFailRate* and *TotalExcRate* which is apparent in the previous Figure 2.

The Lasso-based feature selection method identifies features with a strong linear correlation with the target variable As discussed in Section III-E. This is clear from the fact that things like execution time and age-related features like *Age*, *RecentAvgExeTime*, and *Duration* are given more weight than other factors. These factors will likely have a clear and understandable effect on the outcome. On the other hand, Lasso is renowned for its ability to generate sparse solutions by reducing specific coefficients to zero, essentially carrying out feature selection. As a result, the models focus on a narrower collection of features, specifically those related to execution time, age, and a few others such as *CommitCount* and *LinesAdded*. This technique applies a penalty to the absolute magnitude of the regression coefficients, potentially resulting in the omission of highly correlated features. This could clarify the lack of some characteristics in the PCS-based feature selection.

Finding 4: The feature *LastFailureAge* appears among all the three techniques as the most useful feature to predict failed tests.

B. RQ2: EFFECTIVENESS ON RTP

1) ANALYSIS OF RTP USING ALL FEATURES

To answer the RQ2, we begin with calculating the the APFD values across fifteen datasets for four distinct machine learning models, namely DT, RF, XGBoost, and GBoost using all available feature. The APFD values offers insightful revelations into each model's test prioritization efficiency. This evaluation, leveraging all available features, indicates not only the models' effectiveness in prioritizing fault-detecting test cases but also their performance consistency and variance.

TABLE 7. APFD values for four machine learning models across 15 datasets using all features.

Dataset	DT	RF	XGBoost	GBoost
apache/curator	0.729	0.924	0.928	0.935
apache/rocketmq	0.762	0.893	0.913	0.876
apache/shardingsphere	0.960	0.981	0.987	0.984
apache/sling	0.952	0.978	0.987	0.971
camunda/camunda-bpm-platform	0.905	0.927	0.927	0.924
eclipse@steady	0.860	0.935	0.930	0.844
EMResearch@EvoMaster	0.682	0.880	0.882	0.878
facebook@buck	0.982	0.982	0.986	0.984
Graylog2/graylog2-server	0.736	0.831	0.840	0.825
IBM Open-Liberty	0.749	0.872	0.829	0.851
jcabi/jcabi-github	0.773	0.779	0.779	0.779
JMRI/JMRI	0.780	0.923	0.968	0.806
SonarSource/sonarqube	0.942	0.973	0.978	0.978
yamcs@Yamcs	0.779	0.898	0.912	0.904
zolyfarkas@spf4j	0.743	0.912	0.938	0.802

Table 7 shows that DT has the highest APFD value in only one dataset, while RF and GBoost have the highest APFD values in four cases each. XGBoost, however, has the highest APFD value more often than any other ML model, demonstrating its superior effectiveness in ranking fault-prone test cases.

The RF model's consistent high APFD values, particularly in datasets like apache@shardingsphere and SonarSource@sonarqube where values approached or exceeded 0.98, instills confidence in its versatility for fault detection. While slightly less consistent, the DT model still performed robustly across various contexts, with its highest performance noted in apache@shardingsphere, further highlighting its potential for early fault detection in specific scenarios. The GBoost model performs well, with a top APFD of 0.984 on the apache@shardingsphere dataset, showing its strength in early fault detection. However, its APFD drops to 0.878 on the EMResearch@EvoMaster dataset, indicating variability across datasets.

In contrast, the analysis reveals that XGBoost showcased efficiency in RTP across the datasets. With its highest APFD value reaching 0.987 in the apache@shardingsphere dataset, XGBoost stands out for its capability to effectively prioritize fault-prone test cases. The lowest APFD value observed for XGBoost, at 0.780 in the apache@airavata dataset, still represents a high-efficiency level, far above the previously mentioned range. This wide range of performance shows how flexible and suitable XGBoost is in several different testing environments, pointing to a well-aligned model that can effectively find faults.

The variance in APFD values across different datasets underscores the need for a strategic approach in model selection. The performance of RF and XGBoost, with their maximum APFD value reaching 0.981 and peak at 0.987, demonstrates their robustness and adaptability in different

datasets. However, the observed variance, especially in contexts with lower APFD values, highlights the importance of considering the dataset's intrinsic properties and testing process goals. This ensures the chosen model complements these features and improves fault detection capabilities, emphasizing the critical role of the audience's decision-making process.

Finding 5: **RF** and **XGBoost** models trained with all features, excel in maximizing APFD values, demonstrating robust and adaptable fault detection capabilities across diverse datasets.

2) ANALYSIS OF RTP USING IG TECHNIQUE

From Table 8, we can observe, XGBoost demonstrates high APFD values across all datasets, with its performance peaking at 0.983 in the dataset *facebook@buck* and showcasing strong results in others, such as 0.980 in *apache@shardingsphere* and 0.978 in *SonarSource@sonarqube*. This indicates XGBoost's effectiveness in prioritizing fault-prone test cases, making it a highly reliable model for fault detection in diverse testing environments. Similarly, in several instances, GBoost shows consistent performance, matching or closely following XGBoost's APFD values. It reaches its highest APFD value at 0.985 in *facebook@buck*, suggesting its robust capability in early fault detection across various contexts.

TABLE 8. APFD values for different ML models across datasets using IG feature selection technique.

Dataset	DT	RF	XGBoost	GBoost
apache@curator	0.714	0.877	0.881	0.902
apache@rocketmq	0.738	0.856	0.879	0.848
apache@shardingsphere	0.949	0.974	0.980	0.976
apache@sling	0.942	0.963	0.983	0.966
camunda@camunda-bpm-platform	0.819	0.929	0.914	0.926
eclipse@steady	0.856	0.920	0.901	0.869
EMResearch@EvoMaster	0.702	0.846	0.859	0.859
facebook@buck	0.974	0.980	0.983	0.985
Graylog2@graylog2-server	0.558	0.816	0.784	0.805
IBM@Open-Liberty	0.757	0.809	0.828	0.816
jcabi@jcabi-github	0.767	0.778	0.779	0.779
JMRI@JMRI	0.724	0.811	0.963	0.659
SonarSource@sonarqube	0.945	0.955	0.978	0.978
yamcs@Yamcs	0.777	0.869	0.901	0.895
zolyfarkas@spf4j	0.775	0.848	0.863	0.846

While DT and RF models may not reach the peak values of XGBoost and GBoost, they still perform well in some datasets. For instance, RF demonstrates good performance in *apache@curator* with an APFD of 0.877 and *apache@shardingsphere* with 0.974. While not consistently matching the heights of XGBoost and GBoost across all datasets, this performance is still helpful and contributes to the overall understanding of fault detection.

Finding 6: **XGBoost** and **Gboost** models trained with features with the highest importance, emerge as the consistently better-performing models in terms of APFD.

3) ANALYSIS OF RTP WITH SELECTED FEATURES USING PCA TECHNIQUE

Leveraging PCA for feature selection across various datasets results in a range of APFD values among four machine learning models, as shown in Table 9. This exploration emphasizes PCA's impact on these models' fault detection capabilities.

TABLE 9. APFD values for ML models across datasets using PCA-based features.

Dataset	DT	RF	XGBoost	GBoost
apache@curator	0.7723	0.8129	0.8919	0.8730
apache@rocketmq	0.8059	0.8826	0.869	0.8214
apache@shardingsphere	0.9647	0.9706	0.980	0.9780
apache@sling	0.9559	0.9435	0.9765	0.9771
camunda@camunda-bpm-platform	0.7874	0.8840	0.9113	0.9335
eclipse@steady	0.7964	0.9207	0.9014	0.8616
EMResearch@EvoMaster	0.7531	0.7505	0.8155	0.7984
facebook@buck	0.9746	0.9792	0.9824	0.9809
Graylog2@graylog2-server	0.6642	0.7126	0.7505	0.7853
IBM@Open-Liberty	0.6520	0.7525	0.7270	0.7329
jcabi@jcabi-github	0.7698	0.7712	0.7747	0.7759
JMRI@JMRI	0.8104	0.7826	0.9224	0.6430
SonarSource@sonarqube	0.9488	0.9474	0.9667	0.9667
yamcs@Yamcs	0.8608	0.8243	0.880	0.8821
zolyfarkas@spf4j	0.7770	0.8362	0.859	0.8572

The GBoost model, in particular, stands out with an APFD value reaching 0.9809 in the dataset *facebook@buck* while RF showcases an APFD of 0.8826 APFD value for *apache@rocketmq*, underscoring its consistent strength. XGBoost also demonstrates notable efficacy, with its highest APFD value recorded at 0.9824 for *facebook@buck*, indicating its potent use of PCA for identifying fault-prone tests. This further exemplifies the adaptability and strength of ensemble models in leveraging PCA-transformed features for improved test prioritization. DT exhibits a minimum APFD value of 0.652 in the dataset *IBM@Open-Liberty* and a maximum of 0.9647 in *apache@shardingsphere*. This variation suggests a dependable yet somewhat fluctuating ability to utilize PCA-reduced feature sets across different contexts.

The strategic application of PCA for feature selection fundamentally shifts the operational feature space of these models, emphasizing data components with significant variance. This shift profoundly impacts model performance, as evidenced by the high APFD values in specific models and datasets. However, the variance of APFD values for specific model-dataset combinations indicates potential challenges in consistently applying PCA features across diverse scenarios, highlighting the complex dynamics between model architecture, feature selection technique, and dataset characteristics.

No single ML model has superior performance for PCA, as seen in other cases. This can be attributed to the fact that PCA reduces the dimensionality of the data by focusing on the components that explain the most variance, which may only sometimes align with the specific features that are most predictive for fault detection in every dataset. The effectiveness of PCA can vary significantly depending on how well the principal components capture the relevant

information for the specific testing context, leading to varying results across different models and datasets.

Finding 7: **XGBoost** and **GBoost** models excel in leveraging PCA for feature selection, achieving the highest APFD values and showcasing exceptional efficiency in prioritizing fault-prone tests across diverse datasets,.

4) ASSESSMENT OF RTP WITH FEATURES SELECTED BY LASSO TECHNIQUE

Table 10 showcases APFD values indicates a noteworthy enhancement in fault detection capabilities across the models when Lasso-based feature selection is employed. The APFD values for RF and GBoost models, which frequently surpass the 0.9 mark, reflect a high efficiency in utilizing the refined feature sets for fault detection. For instance, *apache@logging-log4j2* shows exceptional APFD values, with Random Forest achieving 0.987 and Gradient Boosting even slightly higher at 0.986.

TABLE 10. APFD values for ML models across datasets using Lasso-based feature selection.

Dataset	DT	RF	XGBoost	GBoost
apache@curator	0.9089	0.9394	0.9432	0.9415
apache@rocketmq	0.8423	0.9175	0.9207	0.9189
apache@shardingsphere	0.9519	0.9843	0.9856	0.9839
apache@sling	0.9802	0.9753	0.9765	0.9747
camunda@camunda-bpm-platform	0.8696	0.8925	0.8953	0.8930
eclipse@steady	0.7755	0.7889	0.7911	0.7898
EMResearch@EvoMaster	0.7928	0.8061	0.8083	0.8070
facebook@buck	0.9819	0.9950	0.9962	0.9948
Graylog2@graylog2-server	0.8454	0.8587	0.8609	0.8596
IBM Open-Liberty	0.8466	0.8608	0.8620	0.8597
jcabi@jcabi-github	0.7890	0.8023	0.8045	0.8032
JMRI@JMRI	0.8699	0.8832	0.8854	0.8841
SonarSource@sonarqube	0.9723	0.9854	0.9866	0.9852
yamcs@Yamcs	0.8467	0.8600	0.8622	0.8609
zolyfarkas@spf4j	0.7187	0.7320	0.7342	0.7329

DT and XGBoost also demonstrate significant performance with Lasso-selected features, with DT reaching as high as 0.98 in *apache@sling* and XGBoost achieving 0.996 in *facebook@buck*. These values underscore the compatibility of Lasso-based feature selection with various model architectures, enhancing their predictive accuracy and fault detection precision.

The strategic elimination of less significant features through Lasso selection simplifies the models and focuses their learning on the most predictive attributes. This targeted approach is apparent in the improved APFD values, suggesting a direct correlation between feature selection quality and fault detection efficiency. The consistency of high APFD values across multiple datasets also highlights the robustness of Lasso feature selection in improving model performance across diverse software engineering contexts.

Finding 8: Lasso feature selection significantly enhances the fault detection capabilities of ML models, with **RF** and **GBoost** consistently achieving APFD values above 0.9.

C. RQ3

To answer the research question regarding the prediction capability and time efficiency aspects of ML-based feature selection techniques in RTP, focusing on training time and state-of-the-art evaluation metrics such as precision, recall, F1 score, and AUC, we can derive insights from the two tables provided.

1) TRAINING TIME

We compared the training times across datasets using PCA, Lasso, and IG based feature selection techniques with Failed Tests. The results show that PCA and IG technique generally exhibit minimal training times, often close to 0 seconds in several datasets. In contrast, Lasso-based feature selection consistently requires more training time, especially for datasets with a larger number of features or failed tests. For example, in the IBM Open-Liberty dataset, Lasso-based feature selection takes 48.535 seconds, compared to PCA's 6.314 seconds and IG technique's 1.221 seconds. While Lasso provides a thorough feature selection process, it does so at the expense of increased training time, potentially impacting the time efficiency of RTP. However, all the training times in our tests are under a minute, suggesting that the training time for these feature selection techniques is unlikely to significantly impact the overall performance of RTP.

Finding 9: While PCA and IG techniques offer rapid training times, frequently nearing zero, Lasso-based feature selection demands more time, particularly for larger datasets. However, the overall training times are short enough that they are unlikely to impact the performance aspect of RTP significantly.

2) PERFORMANCE METRICS

The analysis of precision, recall, F1-score, and AUC reveals the following insights as shown in Figures 3:

- **Precision** varied from a minimum of 0.76 to a maximum of 0.9950, with an average around the mid-80s percentage across all models and feature selection techniques.
- **Recall** metrics showed a minimum of 0.78 and reached up to 0.9962, with the average recall in the high 80s to low 90s percentage.
- **F1-Score** analysis indicated a minimum value of 0.79, a maximum of 0.9962, and an average in the mid to high 80s percentage.
- **AUC** ranged from 0.83 to 0.9962, with an average indicating excellent model differentiation capability between the classes.

Furthermore, Figures 4 along with the Table 11 presents performance metrics across 15 datasets for four machine learning models using all features, encompassing precision, recall, F1-score, and AUC. The data suggests that all ML models perform competitively across the board with close precision, recall, F1 scores, and AUC values. The DT, RF, XGBoost, and GBoost models exhibit high performance, with F1-scores and AUC values frequently nearing or exceeding 0.9, indicating a strong prediction capability regardless of the feature selection technique that were applied.

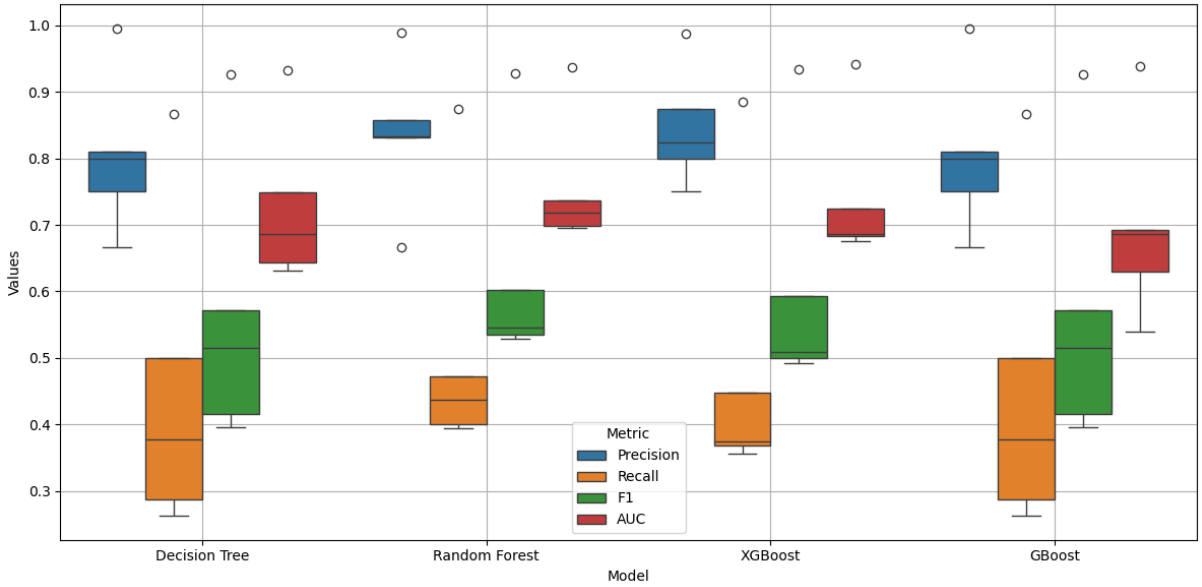

FIGURE 3. Performance metrics (Precision, Recall, F1-Score, AUC) comparison across four ML models.

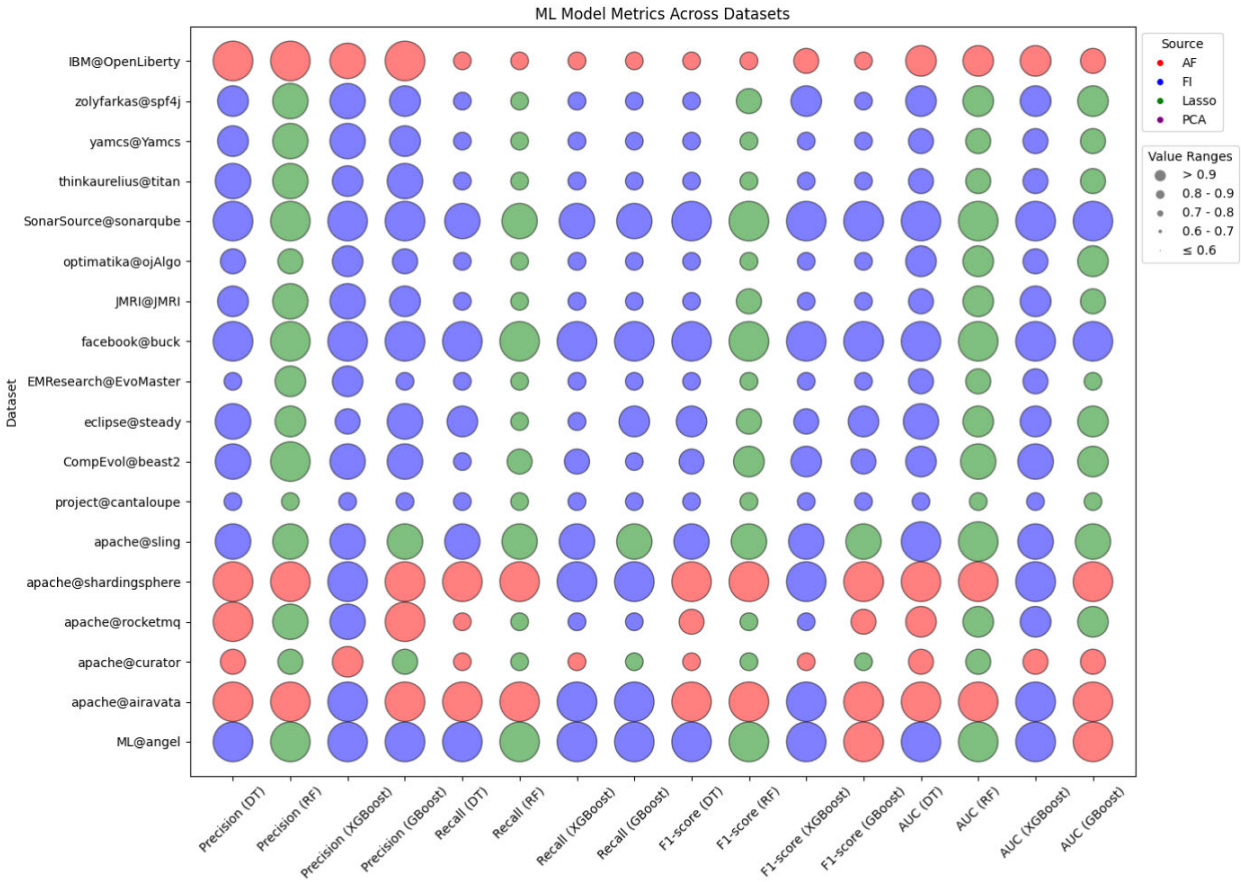

FIGURE 4. Maximum precision, recall, F1-score, and AUC across four ML models for 15 datasets using AF (All Features), FI (IG), PCA, and Lasso techniques.

To simplify the impact of features on model performance, we plot a bubble plot where the x-axis represents four evaluation metrics for each ML model, and the y-axis represents the datasets. The color encoding for four different feature selection techniques is our primary interest in highlighting a theme. The red represents the model trained with all features, and the blue represents the model trained with the IG technique. Finally, green and purple represent models

TABLE 11. Mean and standard deviation of performance metrics across datasets for four ML models.

Model	Precision	Recall	F1-score	AUC
DT	0.943 ± 0.025	0.945 ± 0.025	0.951 ± 0.029	0.939 ± 0.031
RF	0.953 ± 0.029	0.953 ± 0.025	0.960 ± 0.025	0.955 ± 0.025
XGBoost	0.948 ± 0.027	0.956 ± 0.030	0.945 ± 0.031	0.957 ± 0.027
GBosst	0.955 ± 0.022	0.949 ± 0.032	0.959 ± 0.029	0.939 ± 0.028

trained with Lasso and PCA feature selection techniques. The graph shows that approximately 60% of the bubbles are blue, indicating models trained with features based on tree-based importance are consistently providing maximum evaluation metrics. In contrast, approximately 30% of green bubbles, representing Lasso-based features, provide the impact of this technique, followed by 10% of models trained with all features.

The superior performance metrics associated with the IG method underscore its efficacy in selecting relevant features for ML models in RTP. However, the choice of feature selection technique should balance between the predictive performance and training time efficiency, as demonstrated by the comparative analysis of PCA and Lasso techniques.

Our study highlights the significance of selecting appropriate feature selection techniques in enhancing the prediction capabilities of ML models for RTP. IG stands out in maximizing model performance across key evaluation metrics, suggesting its potential as a preferred choice for ML-based RTP applications.

Finding 10: Among the feature selection techniques, **IG-based feature selection technique** consistently provided the highest values across all evaluation metrics, notably in precision, recall, F1-score, and AUC.

D. RQ4: COMPARISON WITH HEURISTICS

This RQ investigates the efficacy of ML models trained with PCA, Lasso, All features, IG, and compares them against traditional heuristic models in the context of fault detection. Through this analysis, we aim to identify the impact of feature selection techniques on improving the models' APFD from simple single feature-based implementations. We have used *FailedRate* as the basis of our heuristic model.

Table 12 reveals several key findings: ML models trained using Lasso and IG feature selection techniques generally outperform heuristic models, indicating that selective feature prioritization significantly enhances fault detection accuracy. Training ML models with all features yields mixed results, suggesting that while comprehensive, the inclusion of all features may introduce noise, which does not always translate to superior performance over heuristic approaches. The PCA-based feature selection technique, despite its ability to reduce dimensionality and focus on variance, does not consistently surpass the heuristic models' performance. This variability points to the method's dependency on the specific characteristics of each dataset. In Table 12, there are three instances where the heuristic model yields the best APFD values: *apache/rocketmq*, *camunda/camunda-bpm-platform*, and *EMResearch/EvoMaster*. To further explore and statistically verify our results, we performed T-tests to compare the APFD values of the Heuristic column with each of the other columns (PCA, Lasso, All features, IG).

A T-test is a statistical hypothesis test used to determine if there is a significant difference between the means of two groups. We used a significance level of 0.05 for our tests. The ''Significance'' column in our table indicates the feature selection techniques for which the Heuristic APFD values are significantly different. For instance, in the case of *apache/curator*, the APFD value of the Heuristic model is significantly different from the APFD value when using IG, as indicated by the entry ''IG'' in the Significance column. This column helps us understand whether the differences we observe are statistically significant, providing a deeper insight into the efficacy of our feature selection techniques.

Finding 11: Effective feature selection through Lasso and IG feature selection significantly enhances fault detection accuracy in machine learning models, outperforming heuristic methods despite their occasional marginal improvements.

E. MONOTONIC AND NON-MONOTONIC EFFECTS, REDUNDANCY, AND RELEVANCY

Our study aimed to enhance RTP by selecting optimal features using machine learning techniques. We analyzed the feature selection process and the resulting model performance through multiple dimensions: monotonic and non-monotonic effects, redundancy, and relevancy. The insights drawn from Tables 7 - 10 illustrate how these aspects interact with different features and techniques.

1) MONOTONIC AND NON-MONOTONIC EFFECTS

a: TECHNIQUE-LEVEL ANALYSIS

From Table 7 to Table 10, we observe both monotonic and non-monotonic effects [48] in the APFD values across different ML models (DT, RF, GBoost, XGBoost) and feature selection techniques (All Features, IG, PCA, Lasso). For DT and GBoost, the APFD values with Lasso-selected features [49] show a consistent improvement across most datasets, indicating a stable and positive relationship. However, the performance with PCA-selected features is inconsistent, with some datasets showing high APFD values and others significantly lower, indicating variability in effectiveness. For RF and XGBoost, the APFD values with

TABLE 12. Comparison of APFD values of ML models trained with PCA, Lasso, all features, IG, and heuristic technique selected features.

Subject	PCA	Lasso	All features	IG	Heuristic	Significance
apache/curator	0.835 ± 0.034	0.808 ± 0.040	0.822 ± 0.088	0.740 ± 0.039	0.8122	IG
apache/rocketmq	0.885 ± 0.027	0.873 ± 0.031	0.883 ± 0.066	0.846 ± 0.029	**0.9061**	All
apache/shardingsphere	0.981 ± 0.009	0.986 ± 0.008	0.986 ± 0.013	0.978 ± 0.005	0.9732	None
apache/sling	0.981 ± 0.006	0.983 ± 0.007	0.972 ± 0.013	0.954 ± 0.006	0.9566	IG
camunda/camunda-bpm-platform	0.927 ± 0.018	0.941 ± 0.021	0.959 ± 0.036	0.961 ± 0.011	**0.9739**	All
eclipse/steady	0.715 ± 0.029	0.703 ± 0.032	0.709 ± 0.038	0.664 ± 0.021	**0.7233**	All
EMResearch/EvoMaster	0.931 ± 0.022	0.940 ± 0.025	0.959 ± 0.041	0.930 ± 0.028	**0.9627**	All
facebook/buck	0.925 ± 0.021	0.941 ± 0.019	0.912 ± 0.025	0.888 ± 0.015	0.8830	None
Graylog2/graylog2-server	0.878 ± 0.015	0.894 ± 0.018	0.871 ± 0.021	0.878 ± 0.005	0.8815	None
IBM Open-Liberty	0.931 ± 0.015	0.942 ± 0.011	0.927 ± 0.009	0.915 ± 0.003	0.8991	None
jcabi/jcabi-github	0.918 ± 0.026	0.937 ± 0.023	0.916 ± 0.054	0.890 ± 0.029	0.8657	None
JMRI/JMRI	0.870 ± 0.012	0.896 ± 0.015	0.889 ± 0.139	0.899 ± 0.022	0.8281	None
SonarSource/sonarqube	0.774 ± 0.008	0.781 ± 0.009	0.776 ± 0.011	0.771 ± 0.000	0.7630	None
yamcs/Yamcs	0.975 ± 0.013	0.982 ± 0.010	0.973 ± 0.014	0.949 ± 0.016	0.9636	None
zolyfarkas/spf4j	0.917 ± 0.018	0.884 ± 0.067	0.957 ± 0.023	0.887 ± 0.019	0.9351	None

IG-selected features are consistently high across datasets, indicating a stable positive relationship. Conversely, the performance with All Features is inconsistent, with high APFD values in some datasets and low in others, indicating variability.

b: FEATURE-LEVEL ANALYSIS

In our feature selection process, we identified monotonic and non-monotonic effects on model performance. Monotonic effects occur when the inclusion of certain features consistently improves the model's performance. For instance, in Table 4, features like *LastVerdict* and *Duration* exhibit monotonic effects, as their inclusion consistently leads to improved APFD values across various datasets, indicating a stable and positive relationship between these features and the model's fault detection accuracy. Conversely, we observe non-monotonic effects with features like *RecentFailRate* in Table 4, demonstrating significant performance fluctuations across different datasets. This variability aligns with the findings of Xue et al. [50], who discuss the challenges of non-monotonic relationships in feature selection.

2) REDUNDANCY

Redundancy in feature selection refers to the presence of multiple features that provide overlapping or similar information, leading to inefficiencies and overfitting. For example, *TotalAvgExeTime* and *RecentAvgExeTime* provide overlapping information about execution times at different intervals and do not appear as important features as in Table 5. To address redundancy, IG-based selection evaluates each feature's contribution to reducing impurity, potentially including redundant features. This is indicated by the APFD value of the selected features in Table 8, as discussed by Nayak et al. [51]. PCA inherently reduces redundancy by transforming correlated features into uncorrelated principal components, as seen in Table 9, where dimensionality is significantly reduced without significant performance loss. Lasso-based selection effectively reduces redundancy by penalizing the coefficients of less relevant features, as highlighted by Xue et al. [50], ensuring that only the most significant features are retained.

3) RELEVANCY

Relevancy in feature selection is about identifying features that significantly impact the model's performance. Different techniques ensure relevancy in various ways. IG-based selection ensures that relevant features are selected based on their statistical contribution to reducing impurity. PCA captures the most variance in the data, ensuring that selected components are appropriate, although it may miss individual feature contributions. Lasso excels in identifying and selecting the most relevant features through regularization, evident from the consistent performance improvements shown in our results, as further emphasized by Chu et al. [49]. Our study shows that certain features are consistently relevant across different selection methods. Features like *LastVerdict* and *Duration* are highly pertinent and contribute significantly to model accuracy across various datasets, as shown in Table 5. Execution time and age-related features are particularly relevant in Lasso-based selection, highlighting their importance in predicting test outcomes.

By integrating the analysis of features and techniques, we observe how each aspect contributes to the overall effectiveness of our feature selection process. Monotonic and non-monotonic effects, redundancy, and relevancy are all crucial factors that interact differently depending on the feature and the selection technique employed. This comprehensive approach provides a robust understanding of optimizing feature selection for improving RTP, addressing the reviewer's concerns about the novelty and thoroughness of our methodology.

V. THREATS TO VALIDITY

- Our choice of hyperparameters could influence the study's outcomes [52], ML algorithms, and feature selection techniques. Correctly tuning these parameters

is paramount for the models' performance, yet exhaustive exploration was beyond this study's scope. We discussed the reasoning behind using the default hyperparameters in subsection III-L.

- Our decision to select tree-based models (RF, DT, GBoost, XGBoost) was driven by their robustness and ability to handle diverse data types and feature interactions, as explained in section III-F. However, other ML algorithms could yield different results.
- While the datasets we used for analysis span diverse scenarios, they may not fully encapsulate the vast array of software testing environments, potentially limiting the findings' applicability across different contexts.
- APFD is widely used as a metric for test case prioritization [42], [53], [54]. However, variations of APFD, such as APFDc and NAPFD [14], [39], which consider additional parameters like time and severity, could alter how evaluation metrics are reported. Our study focused on standard APFD, precision, recall, F1-score, and AUC, which may only capture some nuances of model efficacy in RTP. The study did not use GPU acceleration to expedite calculations, which impacts the overall training time. This decision could affect the scalability and efficiency of the proposed methods, potentially leading to longer training times and increased computational resources required in larger datasets or more complex scenarios.

VI. CONCLUSION

This study investigates the effectiveness of ML-based feature selection techniques on RTP. The findings reveal that in larger datasets, tree-based IG feature selection techniques identify the test verdict of the last build and the failure rate in the previous ten builds, which are the most valuable features for predicting failed test cases while exhibiting the highest fluctuations in importance value. The execution time of the test cases and the test verdict of the last build feature frequently appear as top features across multiple datasets. Most importantly, The number of builds after the last failure is identified as essential by all three techniques: Lasso, PCA, and IG feature selection. On the other hand, XGBoost and GBoost models trained with the top features consistently perform better in APFD, and they excel in leveraging PCA for feature selection. Despite occasional marginal improvements by heuristic models, IG feature selection consistently provides the highest evaluation metrics across various datasets. The strategic application of these techniques can substantially streamline RTP processes, enhancing model accuracy and efficiency. The findings underscore the importance of tailored feature selection, which optimizes predictive performance while aligning with time and resource constraints in contemporary software development environments.

Future work could explore several potential improvements, such as implementing dynamic feature selection methods that adapt to dataset changes over time, combining multiple feature selection techniques to leverage their strengths, and integrating real-time data processing to update IG feature selection and model parameters continuously. Additionally, testing these techniques in a broader variety of datasets and incorporating automated hyperparameter tuning could enhance performance further. Addressing these areas can build on the findings of this study to further improve the effectiveness of ML models in RTP and other related applications.

REFERENCES

[1] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating varying test costs and fault severities into test case prioritization," in *Proc. 23rd Int. Conf. Softw. Eng. (ICSE)*, May 2001, pp. 329–338.

[2] C. Henard, M. Papadakis, M. Harman, Y. Jia, and Y. Le Traon, "Comparing white-box and black-box test prioritization," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. (ICSE)*, May 2016, pp. 523–534.

[3] G. Manogaran and D. Lopez, "A survey of big data architectures and machine learning algorithms in healthcare," *Int. J. Biomed. Eng. Technol.*, vol. 25, nos. 2-4, p. 182, 2017.

[4] M. Khanna, A. Chaudhary, A. Toofani, and A. Pawar, "Performance comparison of multi-objective algorithms for test case prioritization during web application testing," *Arabian J. Sci. Eng.*, vol. 44, no. 11, pp. 9599–9625, Nov. 2019.

[5] Y. Bugayenko, A. Bakare, A. Cheverda, M. Farina, A. Kruglov, Y. Plaksin, W. Pedrycz, and G. Succi, "Prioritizing tasks in software development: A systematic literature review," *PLoS ONE*, vol. 18, no. 4, Apr. 2023, Art. no. e0283838.

[6] M. A. Khan, A. Azim, R. Liscano, K. Smith, Y.-K. Chang, Q. Tauseef, and G. Seferi, "An end-to-end test case prioritization framework using optimized machine learning models," in *Proc. AIST ICST*, 2024, pp. 1–8.

[7] A. S. Yaraghi, M. Bagherzadeh, N. Kahani, and L. C. Briand, "Scalable and accurate test case prioritization in continuous integration contexts," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1615–1639, Apr. 2023.

[8] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, Oct. 2001.

[9] C. Lucchese, C. I. Muntean, F. M. Nardini, R. Perego, and S. Trani, "RankEval: An evaluation and analysis framework for learning-to-rank solutions," in *Proc. 40th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Aug. 2017, pp. 1281–1284.

[10] R. Pan, M. Bagherzadeh, T. A. Ghaleb, and L. Briand, "Test case selection and prioritization using machine learning: A systematic literature review," *Empirical Softw. Eng.*, vol. 27, no. 2, pp. 1–43, Mar. 2022.

[11] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," 2018, *arXiv:1811.04122*.

[12] H. Jahan, Z. Feng, S. M. H. Mahmud, and P. Dong, "Version specific test case prioritization approach based on artificial neural network," *J. Intell. Fuzzy Syst.*, vol. 36, no. 6, pp. 6181–6194, Jun. 2019.

[13] M. Papadakis, L. C. Briand, Y. L. Traon, and M. Harman, "Evaluating the efficiency of change impact analysis for identifying risky software changes," *IEEE Trans. Softw. Eng.*, vol. 43, no. 6, pp. 544–569, Jun. 2017.

[14] J. A. Prado Lima and S. R. Vergilio, "Test case prioritization in continuous integration environments: A systematic mapping study," *Inf. Softw. Technol.*, vol. 121, May 2020, Art. no. 106268.

[15] M. Khatibsyarbini, M. A. Isa, D. N. A. Jawawi, M. L. M. Shafie, W. M. N. Wan-Kadir, H. N. A. Hamed, and M. D. M. Suffian, "Trend application of machine learning in test case prioritization: A review on techniques," *IEEE Access*, vol. 9, pp. 166262–166282, 2021.

[16] B. Ekinci, O. Bozkurt, and M. E. Yilmaz, "A comprehensive study of search-based prioritization techniques," in *Proc. IEEE 27th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2016, pp. 46–57.

[17] S. Shakya and L. C. Briand, "Empirical evaluation of test case prioritization techniques: A replicated study," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, Jun. 2012, pp. 1219–1228.

[18] M. A. Khan, A. Azim, R. Liscano, K. Smith, Y.-K. Chang, S. Garcon, and Q. Tauseef, "Failure prediction using machine learning in IBM websphere liberty continuous integration environment," in *Proc. 31st Annu. Int. Conf. Comput. Sci. Softw. Eng.*, 2021, pp. 63–72.

[19] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," *Softw. Test., Verification Rel.*, vol. 22, no. 2, pp. 67–120, Mar. 2012.

[20] T. Afzal, A. Nadeem, M. Sindhu, and Q. U. Zaman, "Test case prioritization based on path complexity," in *Proc. Int. Conf. Frontiers Inf. Technol. (FIT)*, Dec. 2019, p. 363.

[21] H. Wang, T. M. Khoshgoftaar, and A. Napolitano, "A comparative study of ensemble feature selection techniques for software defect prediction," in *Proc. 9th Int. Conf. Mach. Learn. Appl.*, Dec. 2010, pp. 135–140.

[22] A. Bertolino, A. Guerriero, B. Miranda, R. Pietrantuono, and S. Russo, "Learning-to-rank vs ranking-to-learn: Strategies for regression testing in continuous integration," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 1–12.

[23] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 2836–2856, Aug. 2022.

[24] D. Elsner, F. Hauer, A. Pretschner, and S. Reimer, "Empirically evaluating readily available information for regression test optimization in continuous integration," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2021, pp. 491–504.

[25] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," in *Proc. 26th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, 2017, pp. 12–22.

[26] D. Marijan, A. Gotlieb, and S. Sen, "Test case prioritization for continuous regression testing: An industrial case study," in *Proc. IEEE Int. Conf. Softw. Maintenance*, Sep. 2013, pp. 540–543.

[27] B. Busjaeger and T. Xie, "Learning for test prioritization: An industrial case study," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Nov. 2016, pp. 975–980.

[28] S. Yoo, R. Nilsson, and M. Harman, "Faster fault finding at Google using multi objective regression test optimisation," in *Proc. 8th Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng. (ESEC/FSE)*, Szeged, Hungary, 2011, p. 112.

[29] K. Zhai, B. Jiang, and W. K. Chan, "Prioritizing test cases for regression testing of location-based services: Metrics, techniques, and case study," *IEEE Trans. Services Comput.*, vol. 7, no. 1, pp. 54–67, Jan. 2014.

[30] B. Jiang, Z. Zhang, T. H. Tse, and T. Y. Chen, "How well do test case prioritization techniques support statistical fault localization," in *Proc. 33rd Annu. IEEE Int. Comput. Softw. Appl. Conf.*, vol. 1, Jul. 2009, pp. 99–106.

[31] Y. Jiang and T. Menzies, "Personalized defect prediction," *IEEE Trans. Softw. Eng.*, vol. 39, no. 6, pp. 825–837, 2013.

[32] X. Ling, R. Agrawal, and T. Menzies, "How different is test case prioritization for open and closed source projects?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2526–2540, Jul. 2022.

[33] X. Jin and F. Servant, "CIBench: A dataset and collection of techniques for build and test selection and prioritization in continuous integration," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, May 2021, pp. 166–167.

[34] R. Krishnamoorthi and S. A. Sahaaya Arul Mary, "Factor oriented requirement coverage based system test case prioritization of new and regression test cases," *Inf. Softw. Technol.*, vol. 51, no. 4, pp. 799–808, Apr. 2009.

[35] G. Grano, C. Laaber, A. Panichella, and S. Panichella, "Testing with fewer resources: An adaptive approach to performance-aware test case generation," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2332–2347, Nov. 2021.

[36] S. Murugan, G. Kulanthaivel, and V. Ulagamuthalvi, "Selection of test case features using fuzzy entropy measure and random forest," *Ingénierie des Systèmes d Inf.*, vol. 24, no. 3, pp. 261–268, Aug. 2019.

[37] J. A. Nuh, T. W. Koh, S. Baharom, M. H. Osman, L. Babangida, S. Letchmunan, and S. N. Kew, "Diversity-based test case prioritization technique to improve faults detection rate," *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, no. 6, pp. 927–934, 2023.

[38] T. Menzies, J. Greenwald, and A. Frank, "Data mining static code attributes to learn defect predictors," *IEEE Trans. Softw. Eng.*, vol. 33, no. 1, pp. 2–13, Jan. 2007.

[39] R. Malhotra, "A systematic review of machine learning techniques for software fault prediction," *Appl. Soft Comput.*, vol. 27, pp. 504–518, Feb. 2015.

[40] C. Birchler, N. Ganz, S. Khatiri, A. Gambi, and S. Panichella, "Cost-effective simulation-based test selection in self-driving cars software," *Sci. Comput. Program.*, vol. 226, Mar. 2023, Art. no. 102926.

[41] A. T. Njomou, A. J. B. Africa, B. Adams, and M. Fokaefs, "Msr4ml: reconstructing artifact traceability in machine learning repositories," in *Proc. 2021 IEEE Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Mar. 2021, pp. 536–540.

[42] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Test case prioritization: A family of empirical studies," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 159–182, Feb. 2002.

[43] H. de S. Campos Junior, M. A. P. Araújo, J. M. N. David, R. Braga, F. Campos, and V. Ströele, "Test case prioritization: A systematic review and mapping of the literature," in *Proc. XXXI Brazilian Symp. Softw. Eng.*, 2017, pp. 34–43.

[44] J. Mendoza, J. Mycroft, L. Milbury, N. Kahani, and J. Jaskolka, "On the effectiveness of data balancing techniques in the context of ML-based test case prioritization," in *Proc. 18th Int. Conf. Predictive Models Data Anal. Softw. Eng.*, vol. 1, Nov. 2022, pp. 72–81.

[45] M. I. Prasetiyowati, N. U. Maulidevi, and K. Surendro, "Determining threshold value on information gain feature selection to increase speed and prediction accuracy of random forest," *J. Big Data*, vol. 8, no. 1, p. 84, Dec. 2021.

[46] Z. Zhao, R. Zhang, J. Cox, D. Duling, and W. Sarle, "Massively parallel feature selection: An approach based on variance preservation," *Mach. Learn.*, vol. 92, no. 1, pp. 195–220, Jul. 2013.

[47] A. A. Philip, R. Bhagwan, R. Kumar, C. S. Maddila, and N. Nagppan, "FastLane: Test minimization for rapidly deployed large-scale online services," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, May 2019, pp. 408–418.

[48] T. Hastie, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.

[49] C. C. F. Chu and D. P. K. Chan, "Feature selection using approximated high-order interaction components of the Shapley value for boosted tree classifier," *IEEE Access*, vol. 8, pp. 112742–112750, 2020.

[50] B. Xue, M. Zhang, W. N. Browne, and X. Yao, "A survey on evolutionary computation approaches to feature selection," *IEEE Trans. Evol. Comput.*, vol. 20, no. 4, pp. 606–626, Aug. 2016.

[51] A. Nayak, B. Božić, and L. Longo, "Data quality assessment and recommendation of feature selection algorithms: An ontological approach," *J. Web Eng.*, vol. 22, no. 1, pp. 175–196, Jan. 2023.

[52] M. A. Khan, A. Azim, R. Liscano, K. Smith, Y.-K. Chang, Q. Tauseef, and G. Seferi, "Machine learning-based test case prioritization using hyperparameter optimization," in *Proc. 5th ACM/IEEE Int. Conf. Autom. Softw. Test (AST)*, vol. 35, Apr. 2024, pp. 125–135, doi: 10.1145/3644032.3644467.

[53] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Trans. Softw. Eng.*, vol. 27, no. 10, pp. 929–948, Oct. 2001.

[54] M. Shepperd, G. Kadoda, and S. G. MacDonell, "Does higher complexity mean better effort prediction?" *IEEE Trans. Softw. Eng.*, vol. 38, no. 6, pp. 1408–1418, Nov. 2012.

MD ASIF KHAN (Member, IEEE) received the M.Sc. degree in computer science from the University of Lethbridge, AB, Canada, in 2016. He is currently pursuing a Ph.D. degree in electrical and software engineering with Ontario Tech University, Oshawa, ON, Canada. Since 2020, he has been involved in multiple projects as a Graduate Research Assistant, including a notable project on Machine Learning-Based Test Case Prioritization with IBM; and enhancing cybersecurity measures at Glasshouse Securities Ltd., using AI technologies. His research interests include machine learning, AI in cybersecurity, and autonomous systems.

AKRAMUL AZIM (Senior Member, IEEE) is currently an Associate Professor with the Department of Electrical, Computer, and Software Engineering and the Head of the Real-Time Embedded Software (RTEMSOFT) Research Group, Ontario Tech University, Oshawa, ON, Canada. His research interests include real-time systems, embedded software, software verification and validation, safety-critical software, and intelligent transportation systems. He is a Professional Engineer in Ontario.

RAMIRO LISCANO (Senior Member, IEEE) received the Ph.D. degree in systems design engineering from the University of Waterloo, in 1998. He is a Professor with the Department of Electrical and Software Engineering, Faculty of Engineering and Applied Science, Ontario Tech University, Canada. His research career has spanned across academia, government research institutions, and private industry. He has published more than 185 peer-reviewed journal and conference publications and a co-inventor of 13 patents. His main areas of research interests include mobile and pervasive computing, sensor networks, distributed systems, and software engineering. He is a Senior Member of IEEE Computer and Communications Society and a licensed Professional Engineer with the Professional Engineers of Ontario. He is a member on several conference technical committees related to mobile computing, wireless networking, ubiquitous computing, and software engineering.

KEVIN SMITH is the CI/CD Architect across App Platform covering key products, including Open Liberty, WebSphere Application Server, and WebSphere Automation. With over 25 years' experience with IBM across a wide range of disciplines, including research, development, and consultancy. He has spent the past decade re-inventing how we deliver our software products–utilizing the cloud at extreme scale paired with real-time analytics and cognitive techniques to drive quality into all aspects of our culture and delivery lifecycle.

YEE-KANG (YK) CHANG is a Senior Technical Staff Member with the Application Services Development Team, IBM. He focuses on building developer tools and engaging developers on the latest technologies and innovations.

GKERTA SEFERI is a CI/CD Software Engineer with IBM App Platform, helping to deliver products, such as Open Liberty, IBM WebSphere Liberty, IBM WebSphere Application Server, and Liberty Containers. She has spent the last five years with IBM, creating and maintaining internal tools and automation to run testing and pipelines at extreme scale using short lived cloud machines, building analytics, and AI to help analyses this large volume of results and help spot quality issues before the code is released.

QASIM TAUSEEF is a Software Engineer with IBM WebSphere organization, focusing on developing CI/CD tooling with a data-driven approach providing real-time analytics, which ensure the quality and in turn delivery of Open Liberty.